하흥주

프론트엔드 개발자

포트폴리오

기술 스택

client

- React with Vite, Bun
- TanStack Query, Router, Table, Virtual
- Zod
- Zustand

api

- Express.js
- Prisma, PostgreSQL

deployment

- Railway
- S3, CloudFront

documentation

- Typst

주요 프로젝트

Arbor The Tree - 진도 및 오답 관리 시스템

IUA React UI - 디자인 라이브러리

Terminal To Blog - 자동 업데이트 블로그

Arbor The Tree - 진도 및 오답 관리 시스템

기획 의도

- 강사 재직 시절, 구글 시트와 파이썬을 이용해 진도 및 오답 관리 시스템을 구축
- 해당 시스템을 웹앱으로 제작

사용자 흐름

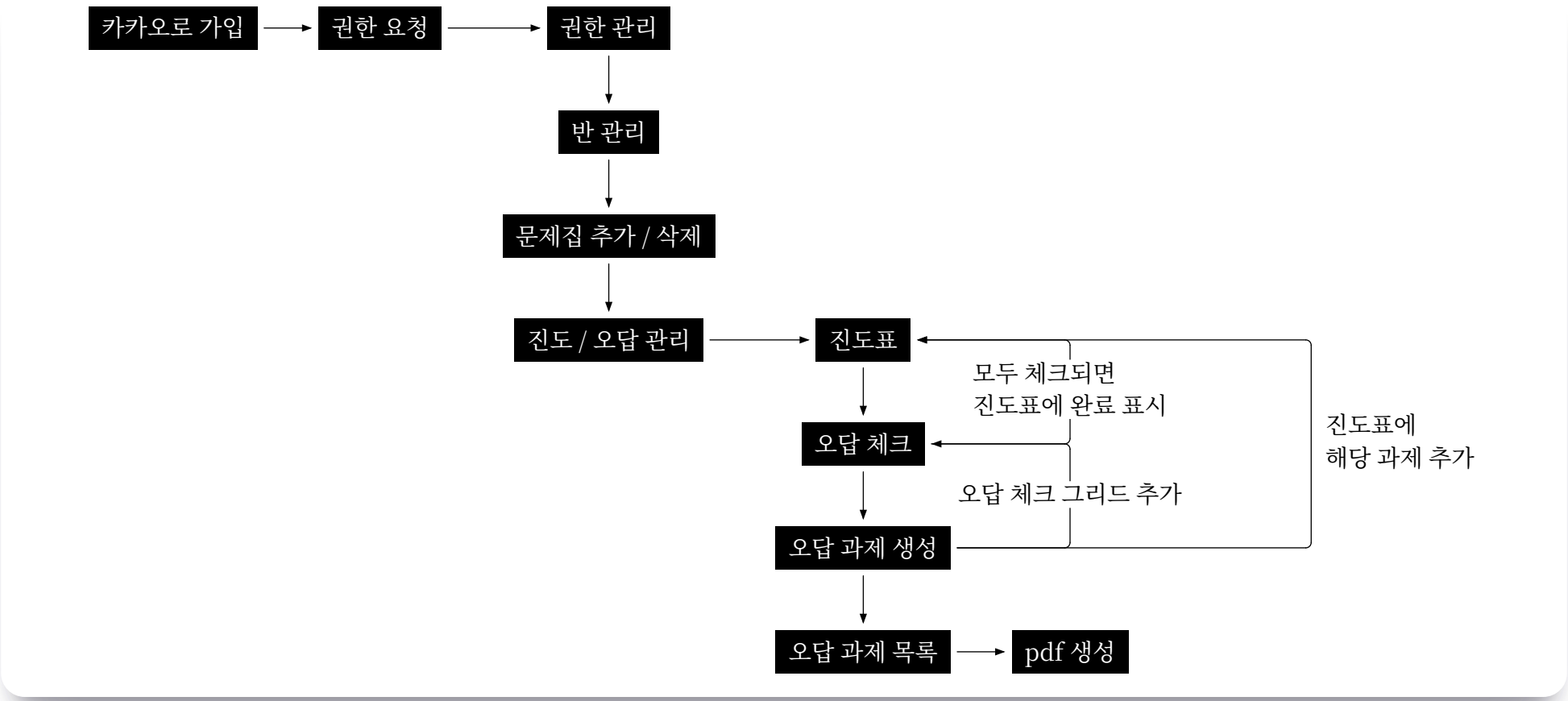

Figure 1: User Flow

Arbor The Tree - UI

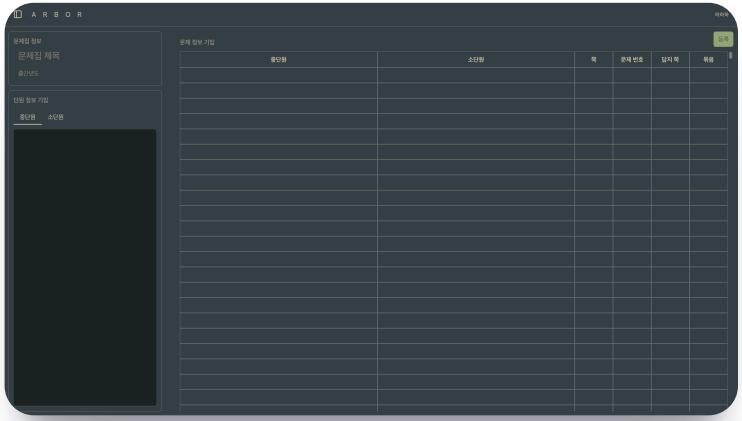

Figure 2: 문제집 추가 / 삭제

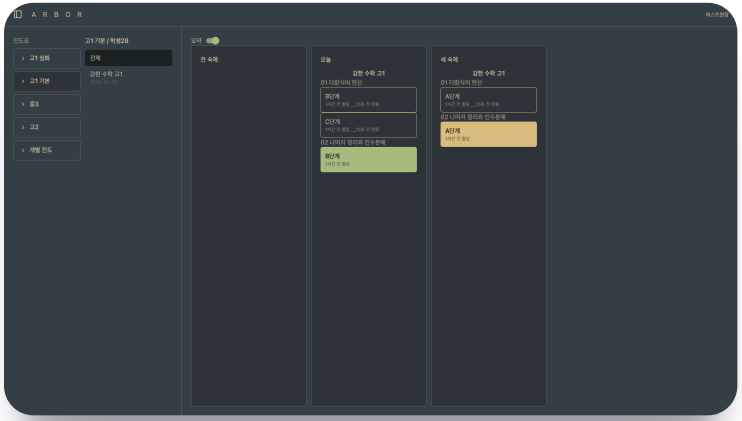

Figure 4: 진도표 페이지

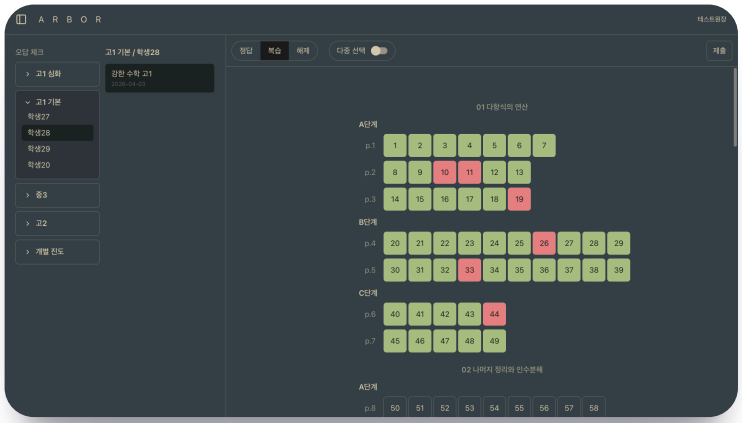

Figure 3: 오답 체크

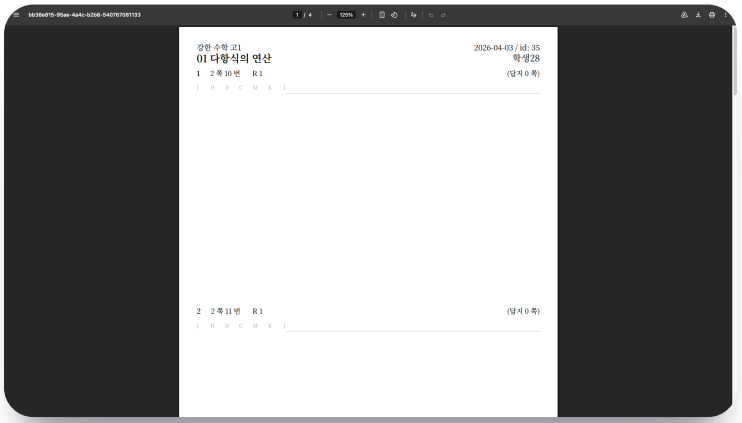

Figure 5: 오답 과제 목록

테스트 계정 이용 안내

Arbor The Tree - 주요 기능

1. 그리드 인풋 및 다중 입력

그리드 인풋

- 구글 스프레드 시트와 유사한 UI
- custom data attribute `data-coordinate`을 이용한 키보드 내비게이션

다중 입력

- 셀에 / 입력 시, 자동으로 알맞는 값을 오버레이, 그 밑의 빈 셀 모두에 같은 값 오버레이
- 문제 번호에 ~ 입력 시, 해당 번호들을 자동 완성 (예: `확인 1~10`)
- 문제 번호 끝에 - 입력 시, 해당 셀부터 끝까지의 문제 번호 삭제 (예: `122-`)

유효성 검사

- 문제 번호 없이 다른 열의 값 기입 시 오류 알림
- 자동 완성할 것이 더 이상 없을 때 /를 사용 시 오류 알림
- 문제 번호가 없는 행에서는 자동 완성 비활성화

해결한 문제

- 빠른 반응을 위해 유효성 검사 셀 수를 최소화하려 하였으나, 해당 로직이 너무 복잡해지고 버그가 많아짐
- 모든 입력마다 전체 셀 유효성 검사를 하되, 가상 스크롤(TanStack Virtual)을 이용해 돔 노드를 최소화하여 20.38ms에서 2.28ms로 반응 시간 89% 단축(`console.time`으로 측정)

Figure 6: Grid Input

문제집 등록 페이지
테스트 계정 이용 안내

Arbor The Tree - 주요 기능

2. 서버 사이드 PDF 생성

최초 계획: 클라이언트 사이드 PDF 생성

- React-pdf를 이용한 클라이언트 사이드 PDF 생성 계획
- 해당 작업이 브라우저 메인 스레드를 차지함
- 때문에 30쪽 이상의 PDF 생성 시, 웹 워커 사용이 권장됨
- 의문: pdf는 브라우저와 상관 없는데 웹 워커까지 써야 하나?

개선안: compile typst to PDF

- request > create typst file > compile to PDF > respond

troubleshooting

- 문제 1: typst는 npm install로 설치 되지 않음
- 문제 2: typst, font 파일은 `tsc`로 빌드되지 않음
- 해결: `railway.json`에 `buildCommand` bash script를 작성
  1. typst compiler 설치
  2. `npm run build`
  3. build에서 제외되는 파일들 dist 폴더로 이동
     - typst template files
     - font

성과

- 8쪽 생성 시간 4.7% 단축
- 80쪽 생성 시간 30.7% 단축
- 800쪽 생성 시간 61.3% 단축
- PDF 생성이 브라우저 메인 스레드를 점유하지 않아 사용자 조작에 반응

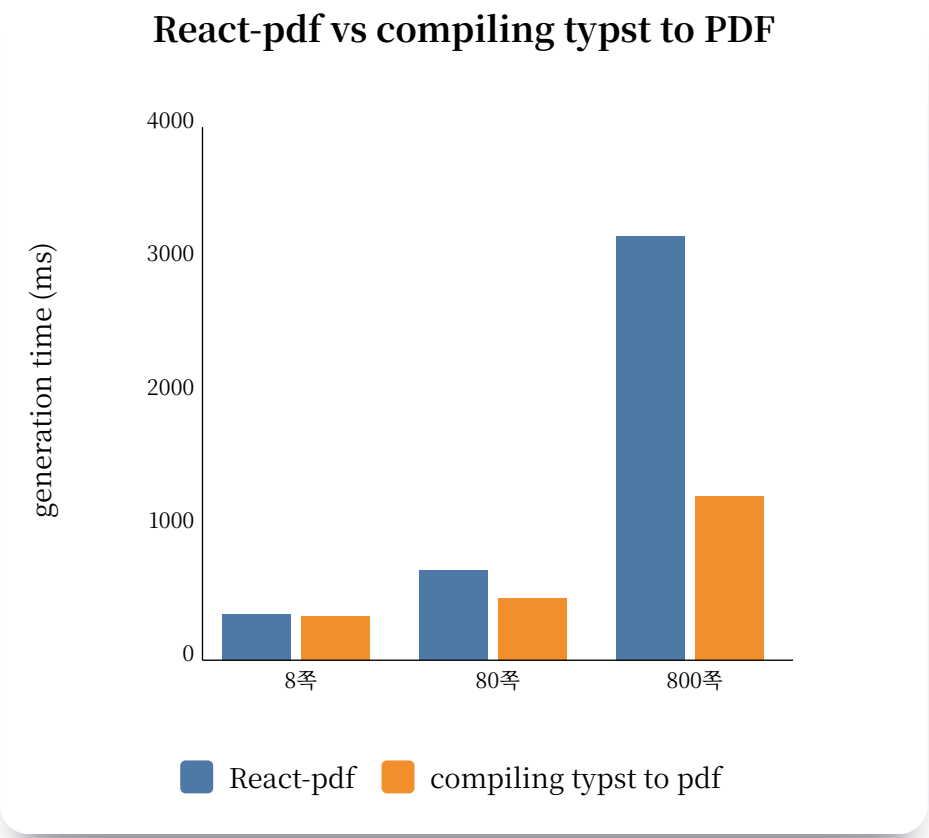

Figure 7: PDF Generation Time Comparison

성능 비교 테스트 페이지

Arbor The Tree - 주요 기능 외 해결한 문제

1. 컴포넌트별 Zustand 스토어

최초 계획: Context API 사용 공통 컴포넌트

- 공통 컴포넌트 개발 중, 상태 전달을 쉽게 하기 위해 Context API를 사용
- boilerplate을 줄이기 위해 zustand로 해당 기능 구현 시도

개선안: 공통 컴포넌트 별 Zustand Store

zustand store: global vs local

how to create global store

- `create` or ...

1. pass values to `createStore`
2. `useStore`

how to create local store

1. pass values to factory function that returns `createStore`
2. call factory function to create distinct stores
3. `useStore` with distinct stores

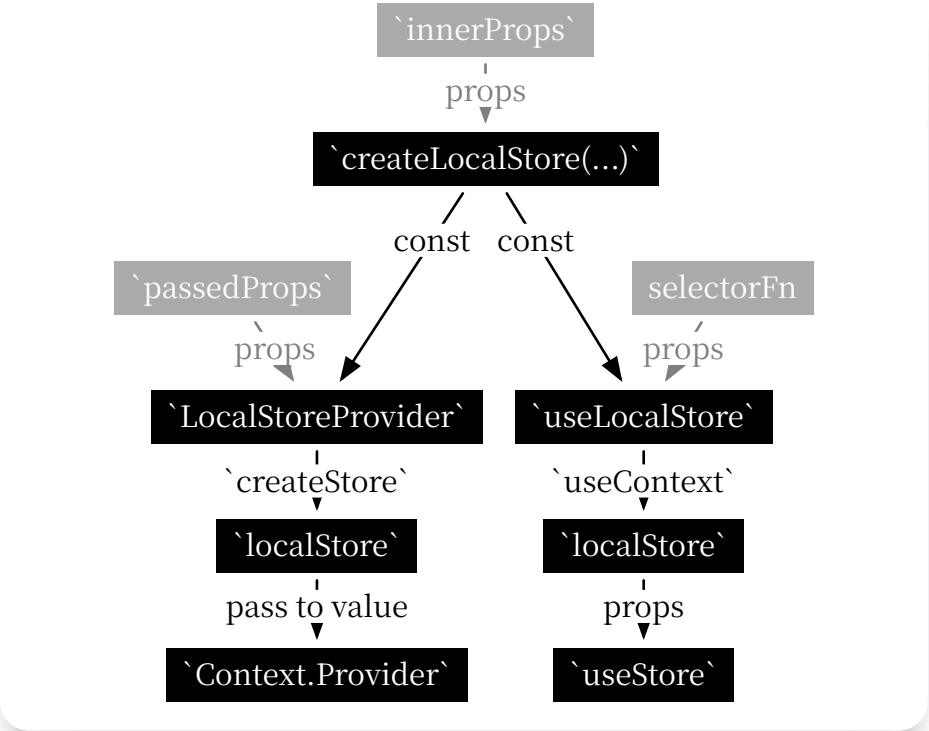

Figure 8: Component-Local Store Pattern

Arbor The Tree - 주요 기능 외 해결한 문제

2. 서버 응답 속도 개선

최초 계획: API In Railway, DB With Neon

- PostgeSQL 서버를 제공하는 Neon을 이용해 DB 구축
- 문제: 문제집 생성할 때 응답까지 7초 가량 걸림
- 원인 후보: network latency, cold start

troubleshooting

- chaned Railway region from “US East (Virginia, USA)” to “South-east Asia (Singapore)”
  - 응답 시간이 여전히 5~7초
- Neon의 free plan은 cold start가 강제됨
- Neon의 region은 “AWS US East 1 (N. Virginia)” 하나 밖에 없음

개선안: deploy my psql, also in railway

- 응답 속도 단축을 위해선
  1. cold start 없는 DB 서버 필요
  2. 최대한 가까운 region에 배포해야
- Railway에 PostgeSQL 서버 배포
  - API와 DB를 같은 internal server에 배포하여 round-trip latency 최소화

성과

- 문제집 생성 시간 315 ms로 단축 (22배 향상)

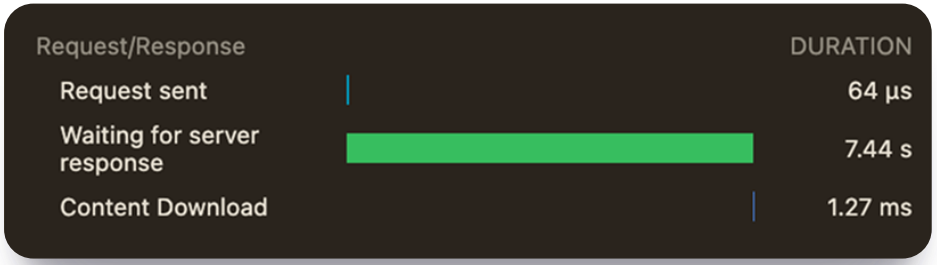

Figure 9: Original Server Response Time

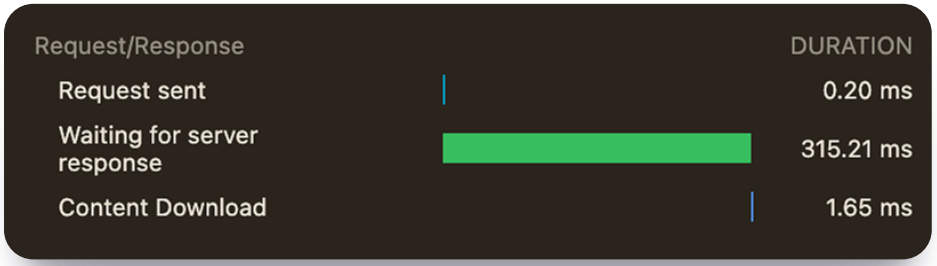

Figure 10: Improved Server Response Time

Arbor The Tree - 이용 안내

링크

- https://arbor-the-tree-production.up.railway.app/
- https://github.com/ThePott/arbor-the-tree
- 테스트 계정 로그인 페이지
- pdf 생성 성능 비교 페이지

테스트 계정

계정	ID	Password
원장	`test12@test.test`	`test1234!@#$`
실장	`test13@test.test`	`test1234!@#$`
학생	`test14@test.test`	`test1234!@#$`